HybridFaaS: Efficient GPU Serverless via Hybrid Autoscaling

Hao Ming[†]
Huazhong University of Science and Technology
Wuhan, Hubei, China
mingh97@hust.edu.cn

Chencheng Ye[*†]
Huazhong University of Science and Technology
Wuhan, Hubei, China
yecc@hust.edu.cn

Xiaofei Liao[†]
Huazhong University of Science and Technology
Wuhan, Hubei, China
xfliao@hust.edu.cn

Hai Jin[†]
Huazhong University of Science and Technology
Wuhan, Hubei, China
hjin@hust.edu.cn

Qiang-Sheng Hua[†]
Huazhong University of Science and Technology
Wuhan, Hubei, China
qshua@hust.edu.cn

Yuanchao Xu[*]
University of California, Santa Cruz
Santa Cruz, California, USA
yxu314@ucsc.edu

Abstract

Serverless computing is widely adopted for its simplicity, autoscaling, and fine-grained billing. However, GPU-based serverless frameworks often inherit CPU-oriented designs, leading to inefficiencies in machine learning (ML) inference. Specifically, the keep-warm policy, effective on CPUs, is suboptimal on GPUs due to rigid hardware binding, and replica-based autoscaling incurs significant memory overhead and frequent model swaps, which are amplified by large machine learning models. To address these problems, we propose HybridFaaS, a hybrid autoscaling strategy that combines pipelined and tensor-parallel scaling to save memory with replica-based scaling for flexibility, reducing model swapping overhead without undue scheduling complexity. Evaluations demonstrate that HybridFaaS significantly outperforms state-of-the-art systems with 80-97% higher throughput and 99-147× lower P99 tail latency.

CCS Concepts: • **Computer systems organization** → **Cloud computing**.

Keywords: Serverless computing, GPU inference, Resource scheduling

[*]Chencheng Ye and Yuanchao Xu are corresponding authors.
[†]Hao Ming, Chencheng Ye, Xiaofei Liao, Hai Jin, and Qiang-Sheng Hua are with National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China.



This work is licensed under a Creative Commons Attribution 4.0 International License.
SIGOPS ATC 2026, Hong Kong, China
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2818-1/2026/11
https://doi.org/10.1145/3828523.3857036

ACM Reference Format:
Hao Ming, Chencheng Ye, Xiaofei Liao, Hai Jin, Qiang-Sheng Hua, and Yuanchao Xu. 2026. HybridFaaS: Efficient GPU Serverless via Hybrid Autoscaling. In *The 2026 ACM SIGOPS Annual Technical Conference (SIGOPS ATC 2026), November 15–18, 2026, Hong Kong, China.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3828523.3857036

1 Introduction

Serverless computing has emerged as a transformative cloud paradigm, offering compelling advantages in programming simplicity, autoscaling, and fine-grained billing mechanisms [4, 5, 18, 43]. The fundamental execution unit, a function encapsulated in a container or microVM, scales automatically to accommodate fluctuating request volumes, eliminating resource overprovisioning and enabling pay-per-use economics. Today, serverless computing permeates diverse workloads including image and video processing [2, 16], machine learning inference [2, 35, 55, 56], and enterprise applications [3, 15], with adoption rates exceeding 70% among AWS users and 50% on Azure and Google Cloud [47].

Prior work has explored diverse approaches to serverless resource management [6–8, 11, 14, 20, 24–27, 37, 38, 41, 42, 44, 46, 48, 49, 51, 55, 58, 63]. However, many frameworks rely on **replica-based autoscaling**, creating or removing independent model replicas and introducing inefficiencies when serving large machine learning models. These limitations arise in two key aspects. First, replica-based scaling increases GPU memory consumption in proportion to the number of replicas. When GPU memory becomes insufficient, GPU-resident model copies must be evicted to make room for other models, leading to repeated model loading from CPU memory even when GPU compute resources remain underutilized. Our measurements show that *model swapping* (transferring a non-GPU-resident model from CPU memory to GPU memory) alone accounts for 21–32% of total execution time in state-of-the-art serverless frameworks

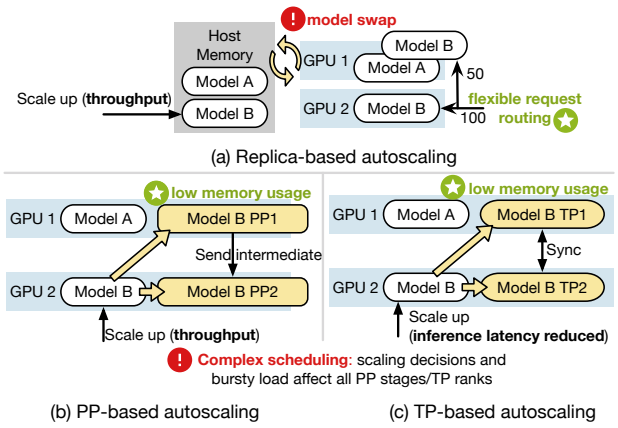


Figure 1. Three types of autoscaling.

under industry workloads (see Section 3), making it a dominant bottleneck. Second, the rapid growth of Large Language Models (LLMs) [12, 50], often with tens of billions of parameters, further intensifies GPU memory pressure and limits the diversity of models that can remain resident in GPU memory. When requests target evicted models, expensive host-to-GPU transfers must complete before inference can begin, complicating scheduling and increasing the operational cost of maintaining service-level objectives (SLOs).

To address the limitations of replica-based autoscaling, we introduce two alternative GPU-aware autoscaling mechanisms by adapting model-parallel execution techniques, originally developed for non-serverless distributed training and inference, to the serverless autoscaling context: **Pipeline-Parallelism-based (PP-based) autoscaling** and **Tensor-Parallelism-based (TP-based) autoscaling**. Although PP and TP have been widely used in distributed ML systems [30, 52, 62], prior serverless frameworks have not fully explored them as dynamic autoscaling mechanisms for improving GPU memory efficiency, reducing model swapping, and meeting latency SLOs under bursty workloads.

Figure 1 illustrates the distinct characteristics of these new autoscaling strategies. Initially, Model A exclusively occupies GPU 1 and Model B exclusively occupies GPU 2; each GPU can hold one full model and half of the other, but not both full models. When Model B's demand exceeds a single GPU's capacity, replica-based autoscaling (Figure 1(a)) adds another full replica on GPU 1, forcing eviction and reloading between Models A and B, resulting in significant swapping overhead. In contrast, PP-based autoscaling (Figure 1(b)) partitions Model B across GPU 1 and GPU 2 to form a pipeline, increasing throughput without model swapping. TP-based autoscaling (Figure 1(c)) partitions Model B across both GPUs, primarily reducing latency to meet strict SLOs, though with limited throughput improvement. These strategies are complementary: replicas are flexible, PP improves memory-efficient throughput scaling but may suffer from *pipeline bubbles* (idle periods when pipeline stages wait for others), and TP reduces latency but incurs cross-device synchronization overhead.

We propose **HybridFaaS**, a GPU serverless system that dynamically combines replica-, PP-, and TP-based autoscaling. Rather than committing to one scaling strategy, HybridFaaS leverages their complementary strengths: replicas provide flexible throughput scaling, PP improves memory-efficient throughput scaling, and TP reduces latency when required by strict SLOs. This hybrid design enables HybridFaaS to adapt to diverse workloads and resource availability while reducing model swapping and improving GPU utilization.

Realizing this design, however, introduces a multidimensional scheduling problem. The scheduler must jointly determine the replica/PP/TP configuration, GPU placement, batch sizes, temporal execution order of colocated models, and scale-up/down decisions under sub-second constraints. This optimization is challenging because each GPU has limited memory and compute capacity, while each model instance has heterogeneous resource demands.

HybridFaaS addresses this challenge with three key techniques. First, it dynamically selects among replica-, PP-, and TP-based autoscaling based on workload characteristics and SLO requirements, using TP only when necessary for latency and relying on replicas or PP for throughput scaling. Second, it introduces a polynomial-time dynamic-programming algorithm to efficiently determine parallelism configurations and GPU placements. Third, it uses load-aware request routing based on estimated sustainable throughput rather than round-robin routing [4, 33], protecting colocated models from interference and enabling effective scale-down.

We implement HybridFaaS on Knative [4] without modifying user functions or requiring specialized hardware. On Azure traces [44], we observe 80–97% higher throughput and 99–147× lower P99 tail latency on average compared with state-of-the-art serverless [55, 58] and ML-serving systems [30]. It also improves GPU utilization by 15–49% and reduces costs by 20–37%.

2 Background of Serverless for ML Serving

Serverless platforms dynamically allocate resources in response to fluctuating workloads through a hierarchical architecture. As shown in Figure 2, a centralized control plane routes inference requests to GPU servers based on resource availability and load-balancing policies. An autoscaler monitors cluster utilization and adjusts capacity by provisioning or reclaiming instances as demand changes. This elasticity enables ML inference workloads to scale automatically while supporting pay-per-use billing.

Several research efforts have explored optimizing serverless infrastructure for ML model serving. INFless [55] employs dynamic batching and heuristic resource assignment to optimize throughput and reduce latency. ServerlessLLM [17]

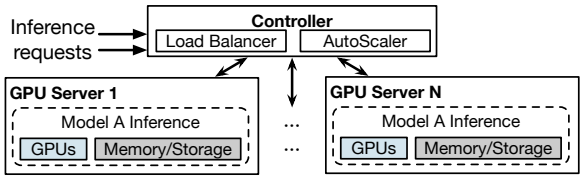


Figure 2. Overview of a GPU serverless cluster.

leverages multi-tier storage and inference migration to mitigate cold start issues. Other studies focus on model partitioning: Gillis [57] automatically splits neural networks into smaller blocks, while Tetris [29] divides models at the tensor level to improve memory efficiency.

3 Motivation

3.1 Parallel Methods for ML Models

ML models primarily use data parallelism (DP, or replica-based methods) [19], pipeline parallelism (PP) [23], and tensor parallelism (TP) [45]. These methods offer different trade-offs in latency, throughput, and memory as the GPU count increases [62], as shown in Figure 3(a–c). DP replicates complete models across GPUs to process independent batches, providing linear throughput scaling with aggregate memory consumption of $O(NM)$, where N is model size and M is replica count. PP divides models into M sequential stages across GPUs, reducing per-GPU memory to $O(N/M)$, but slowest-stage bottlenecks and inter-GPU communication limit throughput scaling to near-linear, slightly below DP. TP partitions individual layers across M GPUs for intra-layer parallel execution, likewise reducing per-GPU memory to $O(N/M)$. Among these strategies, only TP reduces per-request latency, although its substantial synchronization overhead yields only marginal throughput gains.

3.2 Parallel Methods in Serverless

These three parallelism strategies exhibit distinct operational characteristics, each introducing unique challenges when applied to serverless autoscaling with fluctuating request rates on many semi-active instances, as illustrated in Figure 4.

Independent replicas allow replica-based autoscaling to increase throughput while maintaining latency. However, replica-based autoscaling incurs substantial memory consumption, leading to significant model swapping overhead. As illustrated at the top of Figure 4, when a single GPU can accommodate only one model while the system must serve three models under latency constraints, model swapping becomes unavoidable. Initially, models A and C are placed on GPU 1 and GPU 2. When a burst of requests for model B arrives, the GPU memory occupied by models A and C must be freed to load model B. Subsequently, additional swaps are required to process requests for models A and C. These frequent swaps to satisfy each model's latency requirement lead to GPU underutilization and increased cost.

PP-based autoscaling increases throughput while only slightly increasing memory consumption, thereby alleviating the model swapping problem of replica-based autoscaling. However, pipeline bubbles can arise when stage workloads become imbalanced, as overall throughput is constrained by the slowest stage. As illustrated in the figure, partitioning model B across GPU 1 and GPU 2 allows all models to fit in GPU memory, eliminating model swapping. Nevertheless, minimizing pipeline bubbles requires the inference latencies of colocated models (e.g., models A and C) to be closely matched. When the load on model A suddenly increases, the resulting imbalance enlarges the pipeline bubbles and degrades performance.

TP-based autoscaling slightly increases memory usage. However, TP-based autoscaling provides limited throughput improvement and therefore cannot effectively support the autoscaling of serverless systems. Instead, TP primarily reduces the inference latency.

Take away: Consequently, a single parallelism method for serverless autoscaling cannot simultaneously optimize latency, throughput, and memory efficiency across diverse models, dynamic serverless workloads, and resource availability.

3.3 Limitations of Existing Studies

State-of-the-art serverless ML inference systems [1, 4, 17, 33, 55, 58] rely solely on replica-based autoscaling, causing frequent model swapping, GPU underutilization, and increased operational cost. We evaluate INFless [55] and Torpor [58] across multiple models using diverse workloads derived from Azure traces [44] (Section 6). As Figure 3(d) shows, model swapping accounts for 32% and 21% of their total execution time, respectively, on four V100 GPUs under normal workloads.

4 Design

4.1 Overview

HybridFaaS dynamically combines replica-, PP-, and TP-based autoscaling to maximize GPU utilization and throughput while satisfying per-model SLOs. Replicas provide flexible throughput scaling, PP enables memory-efficient scaling, and TP reduces latency for models with tight SLOs. The key challenge is to select the parallelism configuration, GPU placement, batch size, execution order, and scaling action under strict serverless latency constraints. HybridFaaS addresses this challenge using three Principles (**P**):

P1: Minimality of TP. Among the three autoscaling strategies, only TP-based autoscaling reduces per-request inference latency. However, TP-based autoscaling yields marginal throughput gains. Accordingly, HybridFaaS applies TP-based autoscaling only when a model's SLO cannot be met by a single device, and uses the minimum TP degree that satisfies the latency constraint. This principle eliminates

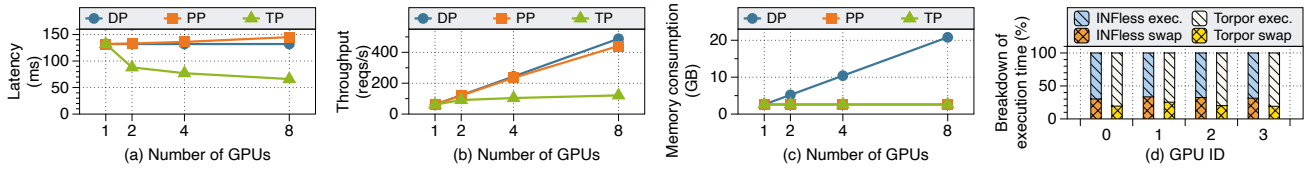


Figure 3. Inference latency (a), throughput (b) and memory consumption (c) when serving BERT-1.3B on NVIDIA V100 GPUs with an input length of 128 tokens and a batch size of 8 under different parallel degrees. Latency and throughput are arithmetic means over five independent runs. (d) Breakdown of the execution time of INFless [55] and Torpor [58]

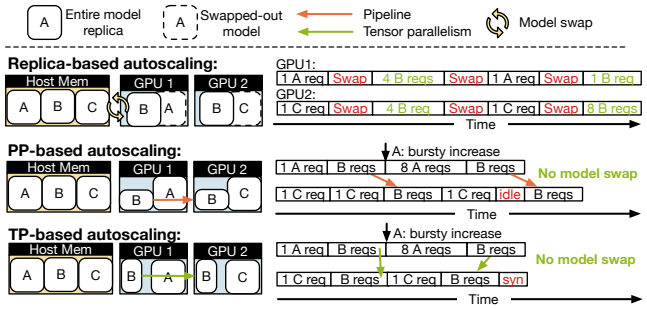


Figure 4. Replica-, PP- and TP-based autoscaling.

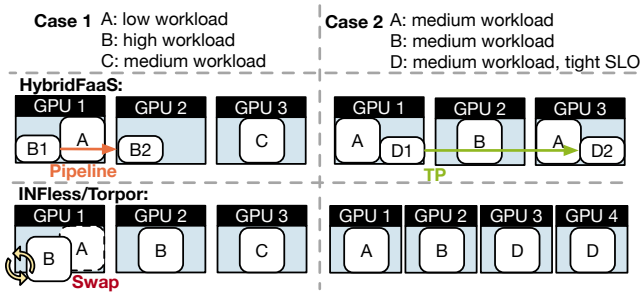


Figure 5. High-Level Illustration Comparing HybridFaaS with INFless/Torpor

TP from the combinatorial search space for the majority of models, substantially reducing scheduling complexity.

P2: Load-balanced pipeline placement. The throughput of a pipelined model is dictated by its slowest stage. To maximize pipeline efficiency, HybridFaaS places successive stages on GPUs whose available throughput capacities are as similar as possible. This insight enables a natural ordering of candidate GPUs by remaining load, which in turn admits an efficient dynamic-programming formulation for the placement search.

P3: Replica flexibility for bursty traffic. While PP- and TP-based autoscaling conserve memory, they couple multiple GPUs into rigid groups that complicate scaling under workload shifts. HybridFaaS retains replicas as the primary mechanism for absorbing traffic bursts and rebalancing load, because replicas are fully independent and can be created, removed, or rerouted without affecting other instances. This complementary use of replicas alongside parallel instances provides the agility required in serverless environments.

Figure 5 presents two representative scenarios that illustrate how these principles translate into concrete placement decisions, compared with the purely replica-based autoscaling of INFless [55] and Torpor [58]. We assume each GPU has 40 GB of memory and each model requires 25 GB.

Case 1. Model B experiences a high request rate that exceeds the throughput of a single GPU, while Models A and C have low and medium request rates, respectively. HybridFaaS partitions Model B into two PP stages across GPU 1 and GPU 2, and temporally shares one stage with the lightly loaded Model A on GPU 1. In this configuration, GPU 1 consumes 37.5 GB of memory — accommodating Model A and half of Model B without triggering any model swap. Temporal sharing between Model B's pipeline stage and Model A further improves GPU utilization. The B1–B2 pipeline already meets Model B's demand, avoiding an unnecessary replica on GPU 2 and leaving GPU memory available for other models. By contrast, INFless and Torpor, relying solely on replica-based autoscaling, cannot fit full replicas of both Models A and B on GPU 1, resulting in frequent model swaps.

Case 2. Model D has a stringent latency SLO, while Models A and B have medium request rates. Following **P1** in Section 4.1, HybridFaaS applies TP to Model D with the minimum degree required to meet its SLO, partitioning it across GPU 1 and GPU 3. Following **P3**, HybridFaaS creates a replica of Model A on GPU 3 to offload traffic from GPU 1, ensuring all models meet their SLO targets. INFless and Torpor, lacking TP support, must allocate two dedicated GPUs with small batch sizes to satisfy Model D's latency constraint, consuming additional resources and leaving fewer GPUs available for Models A and B.

4.2 System Architecture

HybridFaaS extends the monitoring and autoscaling components of the production-grade Knative framework [4], requiring no modifications to user functions or special hardware. Figure 6 depicts the overall architecture, which comprises a centralized control plane and distributed per-GPU local schedulers, and the workflow.

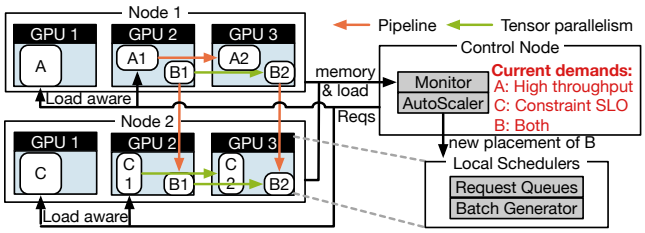


Figure 6. Overview of HybridFaaS.

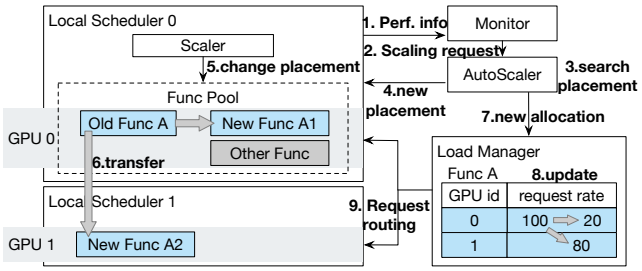


Figure 7. Workflow of autoscaling.

Control Plane. The *monitor* continuously collects per-model and per-GPU telemetry, including request rates, observed throughput and latency, and remaining GPU memory. These measurements are periodically aggregated and provided as input to the *PAController* (Parallel-Aware Controller), the cluster-level autoscaler. Upon each scaling event, the PAController executes the hybrid scale-up (Section 4.4) or scale-down (Section 4.5) algorithm to determine the target DP/PP/TP configuration, GPU placement, maximum batch size, and sustainable throughput for each affected model. The PAController then issues placement and (de)allocation commands to the appropriate local schedulers and updates the load-aware request routing table accordingly.

Local Schedulers. Each GPU is managed by a dedicated local scheduler that maintains a *model pool* in GPU memory, enabling warm starts. A model in the pool may be either a complete replica (for models that fit on a single device) or a partition of a larger model (a PP stage or TP shard). The local scheduler comprises four components: (i) a *performance monitor* that tracks per-instance throughput and latency; (ii) per-model *request queues*; (iii) a *batch generator* that dynamically selects the minimal batch size whose profiled throughput meets the assigned request rate; and (iv) a *PPScaler* responsible for creating, deleting, and migrating model partitions. Upon receiving updated placement decisions from the PAController, the local scheduler adjusts the execution order and batch sizes of colocated instances to minimize pipeline bubbles and TP synchronization overhead.

Communication. Communication between the PAController and local schedulers is facilitated by a per-node *invoker*, which provides system-level services such as model image retrieval during instance creation. This hierarchical design decouples cluster-level placement decisions from device-level execution orchestration, allowing each layer to operate at its natural timescale: the PAController reacts to workload-level shifts on the order of seconds, while local schedulers adapt to request-level dynamics at millisecond granularity.

Autoscaling Workflow. Figure 7 illustrates the autoscaling workflow. The Monitor continuously tracks per-instance performance and issues scaling requests to the AutoScaler whenever it detects insufficient serving capacity. The AutoScaler then determines an effective placement based on current cluster utilization. After deploying the new instance, the AutoScaler updates request routing to balance load across replicas.

4.3 Hybrid Parallelism Configuration

The hybrid autoscaling strategy of HybridFaaS determines the parallelism configuration for a given model in two sequential phases, each guided by a distinct design principle. This two-phase decomposition is critical for tractability: rather than jointly searching over all possible TP×PP×DP configurations, HybridFaaS resolves TP independently and then optimizes over the remaining DP and PP dimensions. Figure 8 illustrates the phases.

For a model whose memory requirements exceed the capacity of a single GPU, HybridFaaS treats its complete PP/TP configuration as one *logical replica*. Each logical replica includes all PP stages and TP shards needed to execute the model, with its weights and runtime state distributed across the participating GPUs. DP-based scaling creates another complete multi-GPU replica, duplicating the model's weights.

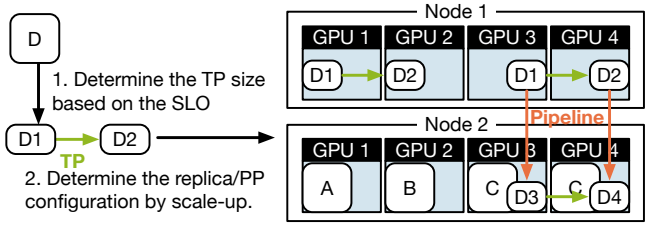


Figure 8. Workflow of setting parallelism configuration.

Phase 1: TP Degree Selection. HybridFaaS first determines whether the model's SLO can be satisfied on a single device. If the profiled single-device inference latency exceeds the SLO, HybridFaaS selects the minimum TP degree t^* such that the profiled TP latency at degree t^* falls within the SLO budget. HybridFaaS avoids increasing TP beyond t^* because higher TP degrees yield diminishing throughput returns due to synchronization overhead. Although increasing TP can also reduce per-GPU memory consumption, PP provides this benefit with better throughput scaling, as shown in Figure 3. HybridFaaS therefore uses the minimum TP degree to

meet the latency requirement and relies on DP and PP for additional throughput. By fixing t^* early, HybridFaaS eliminates the TP dimension from subsequent optimization and treats t^* GPUs as one placement unit. This reduces runtime search overhead, but may exclude jointly optimal TP/DP/PP configurations that use a higher TP degree.

Phase 2: DP and PP Configuration. Given the fixed TP degree, HybridFaaS determines how many replicas to deploy and how deeply to pipeline each replica. The goal is to provision sufficient aggregate throughput to serve the model's current request rate while minimizing memory consumption and avoiding model swapping. Figure 8 illustrates two logical replicas of Model D, both with TP degree 2 but different PP configurations. The right replica consists of two pipeline stages: D1–D2 on Node 1 and D3–D4 on Node 2, each using TP degree 2. Because Model C leaves insufficient GPU memory for a complete TP-only replica on Node 2, PP partitions the right replica across the two nodes to use the remaining memory and improve throughput.

The performance of different DP and PP placements can differ significantly. If a replica has only one stage, it demands excessive memory per device and may trigger model swapping; conversely, a pipeline with too many stages can incur latency bottlenecks, since the slowest stage throttles the entire pipeline.Figure 9 exemplifies these trade-offs under given cluster conditions. In the accompanying table, the *remaining load* represents the maximum additional throughput (in requests per second) of the new model that a GPU can sustain without causing any of its currently running instances to violate their SLOs. In scenario (a), both placement options exceed GPU 2's memory capacity, invoke model swapping, and incur a steep throughput penalty. In scenario (b), although GPU 3 has twice the remaining load of GPUs 1 and 2, its surplus remains idle: the first two stages on GPUs 1 and 2 form a bottleneck that caps pipeline performance. By contrast, scenario (c) maps two pipeline stages to GPUs 1 and 2 — whose loads are well matched—and dedicates GPU 3 to a single stage. This configuration prevents model swapping, exploits GPU 3's extra capacity, and yields the highest throughput. The hybrid autoscaling strategy of HybridFaaS therefore seeks configurations that avoid memory oversubscription and harmonize the throughput of different stages.

A key preparatory step of finding a good DP and PP configuration is to identify candidate GPUs for placement. HybridFaaS sorts the GPUs on each node in descending order of the inference latency of their currently warm models. This sorting ensures that neighboring GPUs in the ranked list host models with similar execution profiles, producing candidate groups that are naturally suited for balanced pipeline placement (per **P2**, Section 4.1). Each candidate group can host either a complete replica or a PP stage of the model being scaled. The actual DP/PP configuration and GPU assignment are then determined by the scale-up algorithm described in

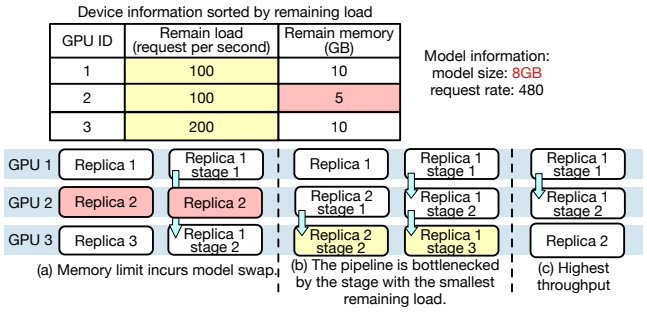


Figure 9. Example of performance differences between various placements. Each single column of replicas represents a placement. The blue arrows denote a pipeline.

Section 4.4, with a time complexity of $O(G^2)$, where G is the number of GPUs in the scheduling partition.

4.4 Scaling Up

A scale-up operation is triggered when the observed request rate for a model exceeds the aggregate sustainable throughput of its current instances. The scale-up strategy embodies two design principles: *saturation before expansion*—first maximize the utilization of already-active GPUs before provisioning new ones, to minimize resource fragmentation—and *balanced pipeline placement*—distribute PP stages across GPUs with similar available capacities, to prevent any single stage from becoming the throughput bottleneck.

4.4.1 Rebalancing. Before searching for a new placement, HybridFaaS first attempts to redistribute the incoming request load across the model's existing replicas. This lightweight adjustment can often handle small load increases—for example, by shifting traffic from a heavily loaded replica to one with remaining capacity—without requiring any resource reallocation. If rebalancing alone proves insufficient to absorb the excess demand, HybridFaaS proceeds to the placement search phase.

4.4.2 Cluster State. The scale-up algorithm relies on two key metrics for each GPU. The *remaining memory*, obtained via standard GPU management utilities (e.g., `nvidia-smi`), determines whether a GPU can physically host an additional model partition. The *remaining load* measures the maximum additional throughput (in requests per second) that the GPU can sustain without causing any of its currently warm instances to violate their SLOs. This metric is computed by examining the *latency slack* of each resident instance—the difference between its SLO deadline and its observed end-to-end latency—and comparing this slack against offline profiling data that maps batch sizes to latency–throughput curves. The resulting throughput value provides a principled upper bound on the additional workload the GPU can absorb.

4.4.3 Dynamic-Programming Formulation. The placement search employs a dynamic-programming recurrence to determine, for each candidate GPU, whether to deploy a new replica, extend an existing replica with an additional pipeline stage, or skip the GPU entirely. Let $max_thr[g][r]$ denote the maximum estimated aggregate throughput achievable when placing the first r replicas of the scaling model across the first g GPUs (sorted in descending order of remaining load). The recurrence is:

$$max_thr[g][r] = MAX\Big\{ \underbrace{max_thr[g-1][r]}_{\text{not place on device g}}, \tag{1}$$

$$\underbrace{max_thr[g-1][r-1] + \Delta_{\mathrm{DP}}(g)}_{\text{add a new replica on device g}}, \tag{2}$$

$$\underbrace{\underset{1 \le j < g}{MAX} \left(max_thr[j][r] + \Delta_{\mathrm{PP}}(j, g) \right)}_{\text{extend a replica with one pipeline stage on device g}} \Big\} \tag{3}$$

the outer MAX evaluates three possible uses of the g-th GPU: **(1) Skip the g-th GPU:** Retain the best placement of r replicas among the first $g-1$ GPUs; **(2) Place a non-pipelined replica on the g-th GPU:** Create a complete, non-pipelined replica on the g-th GPU and add its estimated throughput gain, $\Delta_{\mathrm{DP}}(g)$, to the best placement of $r-1$ replicas among the first $g-1$ GPUs; **(3) Select and extend a replica with one pipeline stage on the g-th GPU:** The j-th GPU is an earlier candidate GPU ($1 \le j < g$) that hosts either a complete replica or a stage of a pipelined replica of the model. The inner MAX enumerates all such GPUs, selects the corresponding replica to extend, and adds one pipeline stage for that replica on the g-th GPU. $\Delta_{\mathrm{PP}}(j, g)$ represents the estimated throughput gain from this choice.

Three properties of this formulation are worth highlighting. First, the descending sort by remaining load ensures that GPUs with similar capacities are adjacent, so pipeline stages naturally land on balanced device groups (**P2**, Section 4.1). Second, by placing instances preferentially on more heavily utilized GPUs, the algorithm consolidates workloads and reduces fragmentation. Third, the formulation reduces the search space from an intractable $O(2^G)$ brute-force enumeration to $O(G^2)$, making it well-suited for real-time execution under strict latency constraints.

When the dynamic programming recurrence identifies a configuration whose aggregate throughput meets or exceeds the target request rate, the scale-up process records the placement. The estimated throughput gain of each newly created instance is preserved and subsequently used as the weight for load-aware request routing.

4.4.4 Load-Aware Request Routing. Effective scaling decisions must be tightly coupled with intelligent request distribution. Unlike conventional serverless platforms [4, 33] that employ round-robin load balancing, HybridFaaS implements a *load-aware routing* policy that exploits the per-replica throughput estimates produced by the scale-up algorithm.

Specifically, the PAController uses each replica's estimated sustainable throughput both as a strict capacity cap and as a proportional weight for load distribution. Incoming requests are routed to temporal-sharing replicas (i.e., replicas colocated with other models) only up to their estimated capacity, with any excess traffic redirected to standalone replicas. For example, if a model spans two replicas with estimated capacities of 100 and 200 requests per second, the PAController distributes traffic in a 1:2 ratio.

This design yields two complementary advantages. First, it protects colocated models from interference: by capping the traffic directed to temporal-sharing instances, the routing policy prevents traffic bursts from overwhelming shared GPUs, preserving the SLO compliance of all resident models. Second, it facilitates efficient scale-down: because standalone replicas preferentially absorb excess load, they are also the first to become idle when demand recedes, making them natural candidates for decommissioning (Section 4.5). This creates a virtuous feedback loop in which scaling decisions and request routing jointly optimize resource utilization.

4.5 Scaling Down

When a model's request rate decreases below the aggregate capacity of its instances, HybridFaaS initiates a scale-down process to reclaim resources. Rather than simply terminating instances, the scale-down algorithm strategically contracts and consolidates model placements to maintain high resource efficiency. The process takes as input the model's current request rate, the throughput and placement of its existing replicas, and the remaining memory and load of the hosting GPUs. It proceeds in three ordered phases.

Phase 1: Decommissioning Idle Replicas. The algorithm first identifies replicas with zero request allocation. These replicas are transitioned to a keep-warm state, in which they remain resident in GPU memory but accept no new requests. If a keep-warm replica remains idle beyond a configurable timeout, it is fully terminated and its GPU memory is released. This two-stage eviction policy balances responsiveness to transient load dips against the cost of cold starts should the model's demand resurge.

Phase 2: Pipeline Contraction. For active replicas, HybridFaaS evaluates whether the pipeline depth can be reduced while still satisfying the allocated request rate. The algorithm checks whether the consolidated stages fit within the memory of a single GPU and whether the resulting throughput suffices. When selecting which stage to remove, the algorithm prioritizes stages running as standalone instances (i.e., not temporally shared with other models), followed by stages on GPUs with the *largest* remaining load. The rationale is that GPUs with the most remaining capacity are the least utilized; removing instances from these GPUs

is most likely to free entire devices, thereby reducing operational cost. This heuristic systematically consolidates workloads onto a smaller set of more heavily utilized devices.

Phase 3: Replica Merging. Finally, HybridFaaS attempts to merge replicas of the same model into a single, deeper pipeline. Replicas are first sorted by their effective throughput. The algorithm then iteratively considers merging adjacent replicas in the sorted order, proceeding only if the resulting pipeline can still satisfy the combined request rate of the merged replicas. Because merging replicas into a pipeline reduces the per-device memory usage (each device holds only a fraction of the model rather than a full copy), memory capacity is not a binding constraint at this phase; the primary consideration is throughput feasibility. Successful merges free GPU memory that can be reclaimed or allocated to other models, further improving cluster-wide resource efficiency.

Figure 10 illustrates the scale-down process with a concrete example. Because the load-aware routing policy (Section 4.4.4) preferentially directs requests to temporal-sharing replicas, HybridFaaS can easily identify the replica with standalone instances—Replica 2 in the figure—and prioritize its contraction. If GPU 3 has sufficient throughput and memory to accommodate Replica 2's entire workload after removing its second stage, the standalone stage on GPU 4 is decommissioned. Replicas 1 and 2 are then evaluated for merging into a three-stage PP, further reducing memory usage.

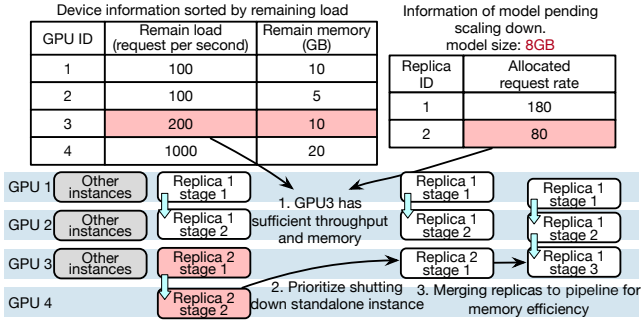


Figure 10. Scaling-down process; blue arrows denote PP.

4.6 Other Complexities

Online Profiling. When offline profiling data is unavailable for a specific model-device pair, HybridFaaS adopts a safe fallback strategy common in production serverless platforms [4, 33] and exploits an online profiling strategy. Specifically, it assigns a conservative default autoscaling threshold and isolates the model instances on dedicated GPUs. From this secure baseline, the autoscaler incrementally increases the parallelism degree while collecting online measurements of latency and throughput. This dynamic, on-the-fly refinement allows HybridFaaS to construct accurate performance profiles without requiring exhaustive offline benchmarking, enabling seamless adaptation to new model deployments or underlying hardware changes.

HybridFaaS also uses scaling feedback to correct profiling errors. Following Knative's default policy, it triggers scale-out when the observed throughput reaches a configurable fraction of the profiled capacity. If SLO violations occur earlier, HybridFaaS treats the profile as overly optimistic and lowers the recorded capacity; if the instance sustains higher load without violations, the capacity is revised upward. This feedback loop makes placement decisions robust to profiling errors and hardware changes.

Scalability. To ensure low scheduling overhead in massive deployments, HybridFaaS scales hierarchically by partitioning the cluster. We partition GPUs into scheduling groups of configurable size, each managed by an independent PAController that restricts placement search to its local group. If a local PAController cannot identify a feasible placement to accommodate a scale-up request within its partition, it seamlessly forwards the excess load to a peer PAController with available capacity. Larger groups provide more placement flexibility but increase search overhead, whereas smaller groups reduce the search space but may increase fragmentation and cross-group forwarding. Independent group scheduling may also miss globally optimal cross-group placements. We use 64 GPUs per group by default, based on the trade-off between placement flexibility and scheduling overhead evaluated in Section 7.5.

5 Implementation

We develop HybridFaaS on top of Knative [4]. The local scheduler has three components: Request Queues, Scaler and Batch Generator. Request Queues are implemented using Flask and Redis (as a queue). Incoming requests are first received through a Flask endpoint and then forwarded to Redis in arrival order.

The batch generator is implemented as a sidecar container, following the common Kubernetes pattern. Based on the assigned request rate and SLO constraints, it forms batches for each colocated model instance and schedules their dispatch times, thereby controlling the execution order among colocated models.

The Scaler is implemented using PyTorch's multiprocessing library and manages GPU resource allocation according to the placement decisions produced by the Autoscaler. During scale-up and scale-down, the Scaler adjusts the parallelism configuration by launching or terminating containers as needed. Each model deployed by the service retains a copy of its weights in CPU memory, which the Scaler uses to load the model when it is not GPU-resident. Releasing a GPU-resident copy therefore requires no weight transfer back to CPU memory. During reconfiguration, HybridFaaS leverages CUDA interprocess communication (IPC) to share

GPU-resident model weights among worker processes in different containers on the same node, and GPU peer-to-peer transfer when model partitions must move across GPUs. This avoids reloading weights from host memory and enables fast model migration without restarting function containers.

We implemented the Autoscaler by leveraging the atomicity of the KNative autoscaler and utilizing Nvidia environment variables. For cross-node scaling processes, Autoscaler simply updates the configuration file of the containers, and KNative autoscaler dynamically scale the related container. This integration ensures both safe and rapid scaling operations. The Autoscaler maintains a per-model routing table that records each instance's placement, parallelism configuration, and estimated sustainable throughput. Local schedulers periodically report observed latency and throughput, which are used to trigger future scaling decisions.

6 Evaluation Methodology

Experimental Infrastructure. The evaluation of HybridFaaS used a cluster with a control node and two computational nodes. By default, each node contains 8 NVIDIA Tesla V100 GPUs (32GB HBM2 memory), connected via NVLink, and 2 Intel Xeon Gold 6130 CPUs. We also evaluate HybridFaaS with nodes equipped with 8 A100 GPUs.

Model setup. We evaluate a diverse set of models spanning domains and computational requirements. We employ large language models from Hugging Face Transformers [53], including BERT variants with 1.3B–15B parameters and Mixture of Experts (MoE) models with 1.3B–7.1B parameters. This range of model sizes enables a thorough assessment of memory demands in ML inference deployments. For computer vision (CV), we use ResNet-50, ResNet-152, and VGG-16 from TorchVision [34].

To establish appropriate SLOs, we measured standalone request latency for each model. CV models were evaluated using standard 224×224 RGB inputs, while NLP models processed 128-token sequences. We defined SLOs following MLPerf [39], and for models not covered there, we adopted SLOs from similarly sized MLPerf models. Table 1 summarizes the model specifications.

Table 1. Evaluated models, their sizes, and their standalone inference latencies (ms).

Model	Size	Latency	Model	Size	Latency
BERT-15B	30GB	189.2ms	MoE-7.1B	14.2GB	33.8ms
BERT-6.7B	13.4GB	85.3ms	MoE-2.4B	4.8GB	14.7ms
BERT-2.7B	5.4GB	47.1ms	MoE-1.3B	2.6GB	13.4ms
BERT-1.3B	2.6GB	23.0ms	VGG16	528MB	3.0ms
ResNet50	98MB	5.5ms	ResNet152	232MB	15.3ms

Workloads. We utilized the Azure Serverless Trace [44], a widely adopted benchmark in contemporary serverless and model-serving research [13, 30, 41, 55, 60]. The trace captures diverse function-invocation patterns, including steady, fluctuating, and bursty demand, allowing us to evaluate HybridFaaS under varying load conditions observed in real-world serverless environments. Following prior methodologies [17, 30, 55], we map each Azure function to our model set in a round-robin manner. This ensures that all models are represented proportionally to their function frequencies in the original trace. To evaluate HybridFaaS under different load conditions, we proportionally scale the trace into normal and high workloads while preserving its temporal patterns. Following prior studies [10, 21, 31, 48], these workloads are calibrated to drive the utilization of our 16 V100 GPUs to approximately 45% and 95%, respectively, representing typical and heavily stressed serving environments.

Baseline. We benchmarked HybridFaaS against three state-of-the-art baseline systems: INFless [55], Torpor [58], and AlpaServe [30]. **INFless** and **Torpor** represent contemporary serverless inference systems implementing replica-based autoscaling with dynamic batching capabilities. **AlpaServe** employs static PP configuration. To ensure a fair comparison, we implemented identical dynamic batching algorithms across all systems. For experimental consistency, we configured identical maximum batch sizes across all systems for each respective model.

Metrics. We use standard serverless metrics: throughput, P99 tail latency, cost, and resource utilization. Cost is computed from the cumulative duration of GPU usage within the cluster. For AlpaServe, which lacks dynamic scale-down, we assume continuous utilization of all GPUs. We also employ a throughput-per-cost metric that combines throughput and cost into an integrated efficiency measure, enabling a more comprehensive assessment of overall system cost-effectiveness in production environments. All throughput and latency results are reported as arithmetic means over five independent runs. For each configuration, we compute the coefficient of variation across five independent runs. The mean coefficients of variation across all evaluated configurations are 3.6%, 4.7%, and 2.8% for throughput, latency, and cost, respectively.

7 Evaluation

7.1 End-to-End Performance

Throughput. HybridFaaS consistently outperforms alternative solutions across all workload scenarios, as illustrated in Figure 11. The system achieves 80%, 74% and 97% higher throughput overall compared to AlpaServe, Torpor, INFless, respectively.

Under high workload conditions, AlpaServe exhibits performance deficiencies stemming from significant interference patterns resulting from its static pipelining strategy when managing concurrent bursty requests. This interference creates resource contention that impacts processing

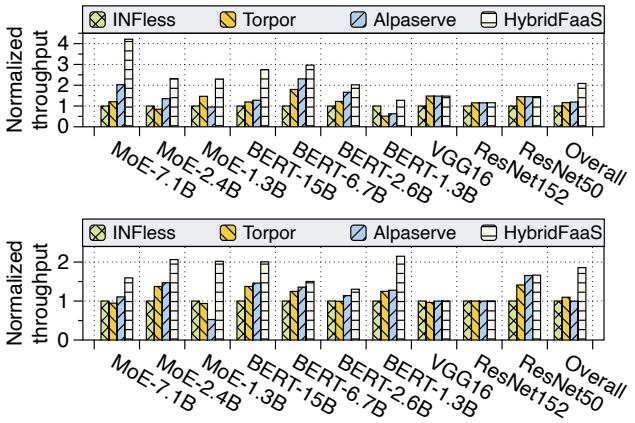


Figure 11. Throughput: high (top), normal (bottom) loads.

efficiency. In contrast, HybridFaaS strategically uses replica-based autoscaling to isolate high-load instances. This architectural design results in more efficient resource utilization and, consequently, higher throughput.

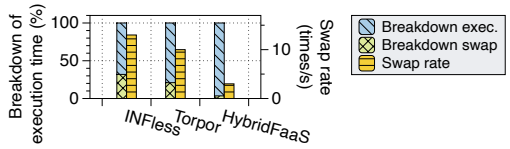


Figure 12. Breakdown of execution time and model swap rates for INFless, Torpor, and HybridFaaS.

INFless and Torpor exhibit markedly lower throughput than HybridFaaS due to limitations in their replica-based autoscaling. When serving heterogeneous workloads, INFless dedicates entire GPUs to low-request-rate models, causing severe GPU fragmentation that starves high-demand models and directly reduces throughput. Their rigid function-allocation strategies also trigger frequent model swapping, further degrading performance. As shown in Figure 12, model swapping accounts for 32% and 21% of total execution time in INFless and Torpor, respectively, versus only 3.5% in HybridFaaS. INFless and Torpor perform 13 and 10 model swaps per second, respectively. By adapting resource allocation to current load, HybridFaaS alleviates memory pressure and reduces these rates by 76% and 70%, respectively.

The performance gap widens under normal workloads, especially for INFless. With only 8 GPUs serving 10 models, INFless cannot uphold its policy of assigning at least one GPU per model, leading to frequent contention and allocation conflicts that severely degrade performance. In contrast, HybridFaaS's hybrid autoscaling maintains higher resource efficiency in such constrained settings, enabling effective model serving even when GPU counts fall below the requirements of replica-based autoscaling.

For some vision models, HybridFaaS demonstrates comparable throughput to the baseline systems. AlpaServe's performance is marginally affected by interference among vision models; however, the relatively low average request rate of vision models limits significant throughput reduction. INFless, despite being affected by frequent model swapping, benefits from the relatively small size of vision models, which enables relatively high peak throughput sufficient to offset the impact of frequent swapping.

Latency. HybridFaaS significantly reduces the tail latency of model requests. Figure 13 demonstrates the P99 tail latency in HybridFaaS, AlpaServe, INFless and Torpor under different loads, normalized to INFless. On average across all models and loads, HybridFaaS reduces P99 tail latency by 99×, 105×, and 147× over AlpaServe, Torpor, and INFless, respectively.

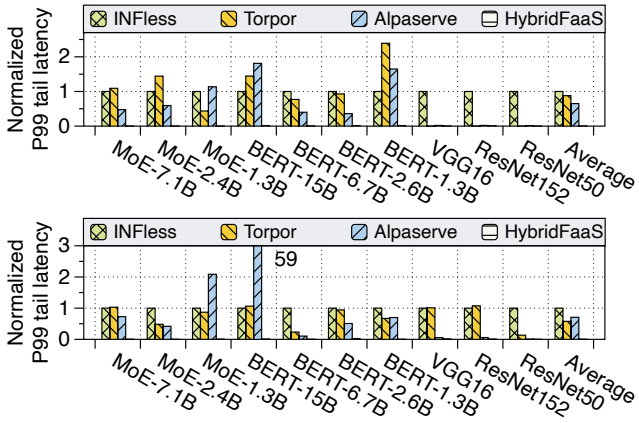


Figure 13. Tail latency: high (top), normal (bottom) loads.

AlpaServe suffers from severe GPU resource contention due to its random colocation strategy, leading to significant latency increases. Additionally, the decreased throughput caused by elevated latency prevents AlpaServe from processing requests in a timely manner, resulting in substantial queuing delays that further exacerbate tail latency. In contrast, HybridFaaS dynamically adjusts placement as request rates fluctuate, avoiding severe interference and leveraging the flexibility of replica-based autoscaling to rapidly rebalance workloads across multiple replicas. Consequently, model latency in HybridFaaS is minimally affected by interference. HybridFaaS also achieves high SLO attainment for large models with strict latency requirements. For BERT-15B (TP=2), HybridFaaS leverages TP to achieve 98% SLO attainment over the entire high-workload trace.

In resource-constrained settings, INFless and Torpor must frequently swap models in and out of GPUs. The high swap-in latency of large models causes severe increases in tail latency, particularly in normal-workload situations with limited GPUs. Moreover, when all instances of a model are swapped out, requests for that model must wait until the

next swap-in operation to execute inference, further exacerbating tail latency. HybridFaaS effectively addresses these challenges by using hybrid autoscaling, conserving GPU memory, reducing model swaps, and increasing the number of warm functions on GPUs.

7.2 Resource Efficiency

Cost Efficiency. We quantify cost using aggregate GPU-hour consumption across all devices. As shown in Figure 14, HybridFaaS reduces cost by 35%, 22% and 37% compared to AlpaServe, Torpor, INFless, respectively. These gains stem from HybridFaaS's hybrid autoscaling strategy that optimizes resource allocation.

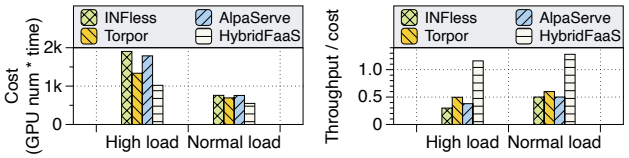


Figure 14. Cost (left) and throughput per cost (right) on high and normal workloads.

In both workload cases, INFless exhibits the highest resource cost, primarily due to time wasted on model swapping. Although Torpor keeps multiple instances warm on a single GPU, it suffers from insufficient memory due to replica-based autoscaling. AlpaServe incurs considerable resource overhead due to severe contention caused by its static pipelining strategy. HybridFaaS, in contrast, maintains a large number of warm functions while promptly identifying effective combinations that maximize throughput and meet SLOs.

Throughput over Cost. To further evaluate HybridFaaS's resource utilization and cost-effectiveness, we compare its throughput-to-cost ratio with baseline systems. As demonstrated in Figure 14, HybridFaaS increases overall throughput per cost by 2.8×, 2.2×, 3.2× compared to AlpaServe, Torpor and INFless, respectively.

Severe resource contention in AlpaServe and frequent model swapping in INFless and Torpor both lead to poor GPU resource efficiency. HybridFaaS leverages flexible function replicas to quickly mitigate interference while keeping more models warm and enabling GPU sharing, thereby alleviating frequent model swapping as shown in Figure 12. As a result, GPU time is devoted primarily to model inference rather than resource-management overhead, substantially improving throughput per cost.

GPU Utilization. HybridFaaS improves average GPU utilization by 39%, 15%, and 49% over AlpaServe, Torpor, and INFless, respectively. Figure 15 shows the average GPU utilization for all systems. To further assess resource efficiency, we plot the cluster-wide GPU utilization CDF. As shown in Figure 16 (right), HybridFaaS's CDF shifts noticeably to the right, indicating more consolidated workloads on fewer active GPUs, higher utilization density, and fewer unnecessarily provisioned GPUs.

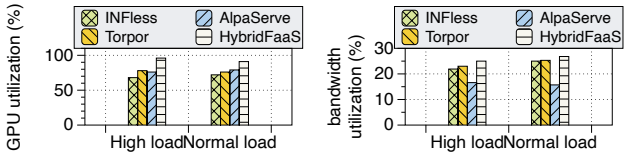


Figure 15. The average GPU utilization (left) and memory bandwidth utilization (right).

The replica-based autoscaling used by INFless and Torpor yields low GPU utilization because each model occupies an entire GPU regardless of its request rate, causing substantial underuse of resources. Frequent model swapping further worsens this inefficiency. HybridFaaS instead uses hybrid autoscaling to colocate an average of 2.75 complete model copies or partitions on each active GPU. This multi-model GPU sharing improves GPU utilization and substantially reduces model swapping frequency.

While AlpaServe distributes models across different device groups, its GPU utilization suffers when request rates within specific device groups are low. Moreover, although AlpaServe may exhibit relatively high GPU utilization, this metric alone does not accurately reflect resource efficiency because colocated models continuously contend for GPU resources. This limitation underscores the importance of our throughput-per-cost metric for comprehensive resource efficiency evaluation.

Memory Bandwidth Utilization. HybridFaaS achieves 30% higher memory bandwidth utilization than all compared solutions, as shown in Figure 15 (right).

SLO Attainment. Figure 16 (left) illustrates SLO attainment as the number of available GPUs varies. HybridFaaS outperforms baseline methods, requiring fewer devices to reach 99% SLO attainment, which further highlights its superior resource efficiency.

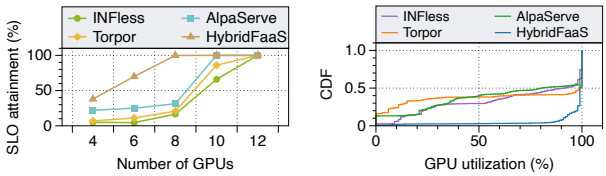


Figure 16. SLO attainment under normal workload trace with varying numbers of GPUs (left), and the CDF of GPU utilization across the cluster (right).

By keeping multiple models with suitable request rates warm on one GPU, HybridFaaS achieves throughput comparable to standalone execution for each model. Even during request bursts, HybridFaaS can effectively rebalance the

workload of multiple replicas. This approach enables HybridFaaS to achieve 99% SLO attainment. In contrast, INFless demonstrates inferior SLO compliance due to GPU resource constraints preventing concurrent instance deployment for all models, necessitating frequent model swapping. Torpor is limited by large memory consumption due to replica-based autoscaling; thus, it still suffers from model swapping. AlpaServe's SLO violations primarily stem from interference during randomly colocated execution of multiple models.

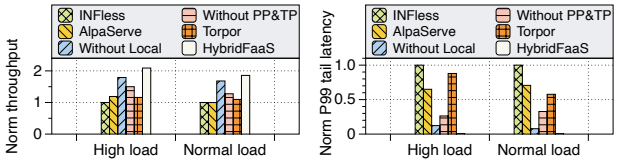


Figure 17. Throughput (left) and P99 tail latency (right) of compared systems, HybridFaaS without TP&PP, and HybridFaaS without Local Scheduler.

7.3 Ablation Study

Figure 17 compares two variants: *Without PP&TP*, which disables PP and TP, and *Without Local*, which disables the Local Scheduler. It demonstrates the effectiveness of HybridFaaS's autoscaling strategies, which significantly enhance performance. PP and TP together contribute substantially to HybridFaaS's throughput by partitioning models to enable model multiplexing on each GPU. As shown in Figure 17, HybridFaaS achieves 1.42× the average throughput of the DP-only configuration (*Without PP&TP*). Runtime statistics further suggest that PP is a major contributor to HybridFaaS's performance gains, as it is the most frequently used scale-up option under the high-workload trace. Scale-up and scale-down account for 75% and 25% of all scaling events, respectively. Among scale-up events, 88% use PP, 19% use DP, and 2% use TP. These percentages overlap because a single decision may combine multiple options.

The Local Scheduler provides an additional performance benefit by selecting execution order and batch sizes to mitigate PP bubbles and TP synchronization overhead. HybridFaaS achieves 1.13× the average throughput of the configuration without the Local Scheduler (*Without Local*).

7.4 Generality

We conducted the same evaluation with 8 Nvidia A100 GPUs, including Qwen3-14B. HybridFaaS improves throughput by 79%, 77% and 99% over AlpaServe, Torpor and INFless respectively, as shown in Figure 18 (left). Thanks to the 600 GB/s bandwidth of the A100 NVLink, the average latency for flushing current requests and migrating KV cache is only 66.6 ms. We also evaluate HybridFaaS on 4 MetaX N260 GPUs, as shown in Figure 18 (right). HybridFaaS achieves 22.9% and 23.0% higher drain-inclusive throughput than our

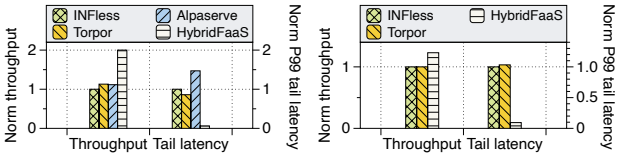


Figure 18. The throughput and P99 tail latency on 8 A100 (80GB) with NVLink (left). The throughput and P99 tail latency on 4 MetaX N260 GPUs on high workload(right)

Torpor and INFless baselines, respectively. It also reduces the normalized P99 latency by 90.9% and 90.6%, respectively.

7.5 Scalability

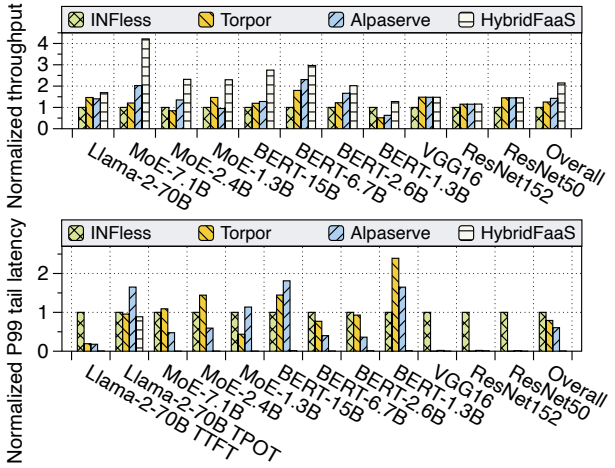


Figure 19. The throughput (top) and P99 latency (down) on a 64-GPU V100 cluster.

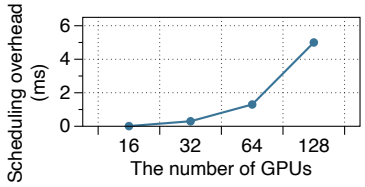


Figure 20. The overhead of PAController searching placement for one model.

Large Model. To evaluate scalability with both more GPUs and a larger model, we evaluated HybridFaaS on a cluster comprising 64 V100 (32GB) GPUs under the high-workload trace, adding Llama 2 70B (TP=4, PP=2) to the evaluated model set. As shown in Figure 19 (left), HybridFaaS achieves 65.9%, 63.4%, and 119% higher overall throughput over AlpaServe, Torpor, and INFless, respectively. For Llama 2 70B specifically, HybridFaaS achieves 20.6%, 15.5%, and 69.2% higher throughput and 23.3×, 25.2×, and 130.5×

lower P99 time to first token than AlpaServe, Torpor, and INFless, respectively. Its time per output token is also 46.3%, 7.2%, and 11.4% lower than the same baselines, respectively.

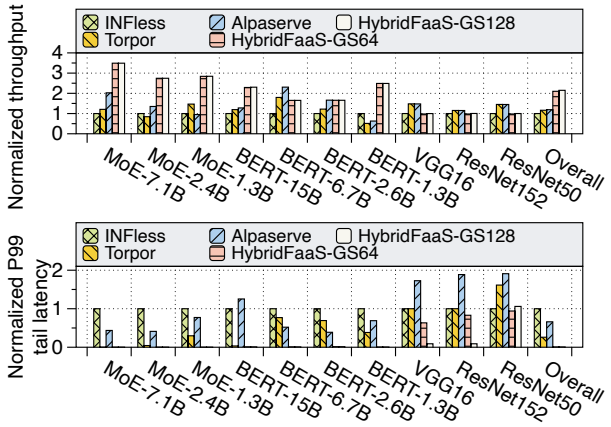


Figure 21. The throughput (top) and P99 latency (down) on a 128-GPU V100 cluster.

Large Cluster. The PAController's placement-search overhead increases with GPU group size. As shown in Figure 20, a single PAController takes approximately 1.3 ms to compute placement for one model in a 64-GPU group, compared with 5 ms in a 128-GPU group. As shown in Figure 21, we further compared two 64-GPU groups ("HybridFaaS-GS64") with one 128-GPU group ("HybridFaaS-GS128") on the same 128 V100 GPU cluster under the high-workload trace. The single 128-GPU group achieved only 2.3% higher aggregate throughput than the two 64-GPU groups. Given this modest throughput gain and the higher scheduling overhead, we use 64 GPUs per group by default.

8 Related Work

Prior work includes serverless-based frameworks and traditional monolithic systems, as shown in Table 2.

Table 2. Related work.

System	Framework	Tensor scaling	Replica scaling	Pipe scaling
INFless [55]	Serverless	No	Yes	No
Torpor [58]	Serverless	No	Yes	No
Batch [1]	Serverless	No	Yes	No
ServerlessLLM [17]	Serverless	No	Yes	No
HydraServe [32]	Serverless	No	Yes	No
BlitzScale [59]	Serverless	No	Yes	No
Aegaeon [54]	Serverless	No	Yes	No
Gillis [57]	Serverless	No	Yes	No
Tetris [29]	Serverless	No	Yes	No
INFaaS [40]	Serverless	No	Yes	No
Orion [49]	Monolithic	No	No	No
AlpaServe [30]	Monolithic	No	No	Static
Choi et al. [9]	Monolithic	No	Yes	No
REEF [22]	Monolithic	No	No	No
HybridFaaS	Serverless	Dynamic	Yes	Dynamic

Serverless resource management: Several studies [1, 17, 29, 55, 57] aim to optimize ML inference in serverless systems. INFless [55] and Batch [1] improve throughput through dynamic batching, while ServerlessLLM [17] and HydraServe [32] focus on reducing cold-start latency and BlitzScale [59] enables fast and dynamic model autoscaling by leveraging model parameters already cached on GPUs and distributing them through network-based multicast. To address the limited memory footprint typically allocated to serverless functions (e.g., 4GB), Gillis [57] and Tetris [29] employ tensor partitioning and tensor sharing techniques, respectively. However, these systems rely on replica-based methods, leading to low GPU utilization and limiting the number of functions that can remain warm.

ML inference serving: Prior ML inference serving systems [9, 22, 28, 49, 61] improve throughput via model colocation. Orion [49], REEF [22], and Laius [61] assume a mix of latency-sensitive and best-effort workloads and colocate best-effort functions to increase GPU utilization. However, this assumption does not hold in serverless environments, where all models typically have SLO requirements, making such colocation prone to interference. Choi et al. [9] and MISO [28] employ NVIDIA MPS [36] to partition GPU resources for colocation, but their scalability is limited by GPU memory capacity. In contrast, HybridFaaS adopts hybrid autoscaling, leveraging pipeline and tensor parallelism to reduce model memory footprints while using replica-based scaling to flexibly separate instances, improving overall resource efficiency.

9 Conclusion

This paper introduced HybridFaaS, a novel serverless framework that resolves the critical inefficiencies plaguing GPU-based ML inference by unifying pipelined, replica-based and tensor-parallel autoscaling into a cohesive hybrid autoscaling strategy. This approach harmonizes the memory efficiency of pipeline and tensor parallelism—essential for large models—with the scheduling flexibility of replication, which is critical for handling bursty, heterogeneous workloads. Comprehensive evaluation demonstrates that HybridFaaS substantially outperforms leading serverless and ML-serving systems, achieving 80-97% higher throughput, 99-147× lower P99 tail latency, and 20–37% lower operational costs.

Acknowledgments

We thank the ATC reviewers for their valuable comments. This work is supported by the National Key Research and Development Program of China (No. 2025YFB4506503), Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118), the National Natural Science Foundation of China (NSFC-RGC No. 62461160333), and CCF-BAIDU (No. OF202512).

References

[1] Ahsan Ali, Riccardo Pinciroli, Feng Yan, and Evgenia Smirni. 2020. Batch: machine learning inference serving on serverless platforms with adaptive batching. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.

[2] Lixiang Ao, Liz Izhikevich, Geoffrey M. Voelker, and George Porter. 2018. Sprocket: A serverless video processing framework. In *Proceedings of the ACM Symposium on Cloud Computing*. 263–274.

[3] Shriram Athreya, Sanmith Kurian, Afrin Dange, and Suvarna Bhatsangave. 2022. Implementation of Serverless E-Commerce Mobile Application. In *2022 2nd International Conference on Intelligent Technologies (CONIT)*. IEEE, 1–5.

[4] The Knative Authors. 2024. *Knative is an Open-Source Enterprise-level solution to build Serverless and Event Driven Applications.* https://knative.dev/docs/

[5] Microsoft Azure. 2022. *Azure Functions.* https://learn.microsoft.com/en-us/azure/azure-functions/functions-overview

[6] Vivek M. Bhasi, Jashwant Raj Gunasekaran, Prashanth Thinakaran, Cyan Subhra Mishra, Mahmut Taylan Kandemir, and Chita Das. 2021. Kraken: Adaptive Container Provisioning for Deploying Dynamic DAGs in Serverless Platforms. In *Proceedings of the ACM Symposium on Cloud Computing*. 153–167.

[7] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. 2020. SEUSS: skip redundant paths to make serverless fast. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–15.

[8] Ruobing Chen, Haosen Shi, Yusen Li, Xiaoguang Liu, and Gang Wang. 2023. OLPart: Online Learning based Resource Partitioning for Colocating Multiple Latency-Critical Jobs on Commodity Computers. In *Proceedings of the Eighteenth European Conference on Computer Systems*. 347–364.

[9] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving Heterogeneous Machine Learning Models on Multi-GPU Servers with Spatio-Temporal Sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 199–216.

[10] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. 2017. Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*. 153–167.

[11] Nilanjan Daw, Umesh Bellur, and Purushottam Kulkarni. 2020. Xanadu: Mitigating cascading cold starts in serverless function chain deployments. In *Proceedings of the 21st International Middleware Conference*. 356–370.

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 https://arxiv.org/abs/1810.04805

[13] Dong Du, Qingyuan Liu, Xueqiang Jiang, Yubin Xia, Binyu Zang, and Haibo Chen. 2022. Serverless computing on heterogeneous computers. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 797–813.

[14] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. 2020. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 467–481.

[15] Sadjad Fouladi, Francisco Romero, Dan Iter, Qian Li, Shuvo Chatterjee, Christos Kozyrakis, Matei Zaharia, and Keith Winstein. 2019. From Laptop to Lambda: Outsourcing Everyday Jobs to Thousands of Transient Functional Containers. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 475–488.

[16] Sadjad Fouladi, Riad S. Wahby, Brennan Shacklett, Karthikeyan Vasuki Balasubramaniam, William Zeng, Rahul Bhalerao, Anirudh Sivaraman, George Porter, and Keith Winstein. 2017. Encoding, Fast and Slow: Low-Latency Video Processing Using Thousands of Tiny Threads. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 363–376.

[17] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 135–153.

[18] Google. 2022. *Google Cloud Functions.* https://cloud.google.com/functions

[19] Priya Goyal, Piotr Dollár, Ross B. Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2018. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour. arXiv:1706.02677 https://arxiv.org/abs/1706.02677

[20] Jashwant Raj Gunasekaran, Prashanth Thinakaran, Nachiappan C. Nachiappan, Mahmut Taylan Kandemir, and Chita R. Das. 2020. Fifer: Tackling Resource Underutilization in the Serverless Era. In *Proceedings of the 21st International Middleware Conference*. 280–295.

[21] Jing Guo, Zihao Chang, Sa Wang, Haiyang Ding, Yihui Feng, Liang Mao, and Yungang Bao. 2019. Who limits the resource efficiency of my datacenter: an analysis of Alibaba datacenter traces. In *Proceedings of the International Symposium on Quality of Service* (Phoenix, Arizona).

[22] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale Preemption for Concurrent GPU-accelerated DNN Inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 539–558.

[23] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc.

[24] Xinning Hui, Yuanchao Xu, Zhishan Guo, and Xipeng Shen. 2024. ESG: Pipeline-Conscious Efficient Scheduling of DNN Workflows on Serverless Platforms with Shareable GPUs. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '24)*. Association for Computing Machinery, 42–55. doi:10.1145/3625549.3658657

[25] Xinning Hui, Yuanchao Xu, and Xipeng Shen. 2025. Exploring Function Granularity for Serverless Machine Learning Application with GPU Sharing. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 1, Article 6 (March 2025), 28 pages. doi:10.1145/3711699

[26] Xinning Hui, Yuanchao Xu, and Xipeng Shen. 2025. FluidFaaS: A Dynamic Pipelined Solution for Serverless Computing with Strong Isolation-based GPU Sharing. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '25)*. Association for Computing Machinery, 18:1–18:14. doi:10.1145/3731545.3731580

[27] Kostis Kaffes, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2022. Hermod: principled and practical scheduling for serverless functions. In *Proceedings of the 13th Symposium on Cloud Computing*. 289–305.

[28] Baolin Li, Tirthak Patel, Siddharth Samsi, Vijay Gadepally, and Devesh Tiwari. 2022. MISO: exploiting multi-instance GPU capability on multi-tenant GPU clusters. In *Proceedings of the 13th Symposium on Cloud Computing*. 173–189.

[29] Jie Li, Laiping Zhao, Yanan Yang, Kunlin Zhan, and Keqiu Li. 2022. Tetris: Memory-efficient Serverless Inference through Tensor Sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA. https://www.usenix.org/conference/atc22/presentation/li-jie

[30] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 663–679.

[31] Qixiao Liu and Zhibin Yu. 2018. The Elasticity and Plasticity in Semi-Containerized Co-locating Cloud Workload: a View from Alibaba Trace. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '18)*. 347–360.

[32] Chiheng Lou, Sheng Qi, Chao Jin, Dapeng Nie, Haoran Yang, Yu Ding, Xuanzhe Liu, and Xin Jin. 2026. HydraServe: Minimizing Cold Start Latency for Serverless LLM Serving in Public Clouds. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, Renton, WA, 415–430. https://www.usenix.org/conference/nsdi26/presentation/lou

[33] OpenFaaS Ltd. 2022. *OpenFaaS*. https://www.openfaas.com/

[34] TorchVision maintainers and contributors. 2016. *TorchVision: PyTorch's Computer Vision library*. https://github.com/pytorch/vision

[35] Hao Ming, Tingting Pan, Dong Chen, Chencheng Ye, Haikun Liu, Liting Tang, Xiaofei Liao, and Hai Jin. 2023. VIDGCN: Embracing input data diversity with a configurable graph convolutional network accelerator. *Journal of Systems Architecture* 141 (2023), 102924.

[36] NVIDIA Corporation. 2026. Multi-Process Service (MPS). https://docs.nvidia.com/deploy/mps/index.html.

[37] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2018. SOCK: Rapid Task Provisioning with Serverless-Optimized Containers. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 57–70.

[38] Haoran Qiu, Weichao Mao, Archit Patke, Chen Wang, Hubertus Franke, Zbigniew T. Kalbarczyk, Tamer Başar, and Ravishankar K. Iyer. 2022. SIMPPO: a scalable and incremental online learning framework for serverless resource management. In *Proceedings of the 13th Symposium on Cloud Computing*. 306–322.

[39] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf inference benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 446–459.

[40] Francisco Romero, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. INFaaS: Automated Model-less Inference Serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 397–411.

[41] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. 2022. IceBreaker: warming serverless functions better with heterogeneity. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 753–767.

[42] Ghazal Sadeghian, Mohamed Elsakhawy, Mohanna Shahrad, Joe Hattori, and Mohammad Shahrad. 2023. UnFaaSener: Latency and Cost Aware Offloading of Functions from Serverless Platforms. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 879–896.

[43] Amazon Web Services. 2024. *AWS Lambda*. https://aws.amazon.com/lambda/

[44] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. 205–218.

[45] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. *CoRR* abs/1909.08053 (2019). arXiv:1909.08053 http://arxiv.org/abs/1909.08053

[46] Won Wook Song, Taegeon Um, Sameh Elnikety, Myeongjae Jeon, and Byung-Gon Chun. 2023. Sponge: Fast Reactive Scaling for Stream Processing with Serverless Frameworks. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 301–314.

[47] Mike Stemle. 2024. *The state of serverless*. https://www.datadoghq.com/state-of-serverless/

[48] Jovan Stojkovic, Tianyin Xu, Hubertus Franke, and Josep Torrellas. 2023. MXFaaS: Resource Sharing in Serverless Environments for Parallelism and Efficiency. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.

[49] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, fine-grained GPU sharing for ML applications. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 1075–1092.

[50] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 https://arxiv.org/abs/2505.09388

[51] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. 2021. Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 559–572.

[52] Yujie Wang, Shenhan Zhu, Fangcheng Fu, Xupeng Miao, Jie Zhang, Juan Zhu, Fan Hong, Yong Li, and Bin Cui. 2025. Spindle: Efficient Distributed Training of Multi-Task Large Models via Wavefront Scheduling. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1139–1155.

[53] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 38–45.

[54] Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyuan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, and Jingren Zhou. 2025. Aegaeon: Effective GPU Pooling for Concurrent LLM Serving on the Market. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) *(SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 1030–1045.

[55] Yanan Yang, Laiping Zhao, Yiming Li, Huanyu Zhang, Jie Li, Mingyang Zhao, Xingzhen Chen, and Keqiu Li. 2022. INFless: a native serverless system for low-latency, high-throughput inference. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 768–781.

[56] Shanhe Yi, Zijiang Hao, Qingyang Zhang, Quan Zhang, Weisong Shi, and Qun Li. 2017. LAVEA: latency-aware video analytics on edge computing platform. In *Proceedings of the Second ACM/IEEE Symposium on Edge Computing*. 1–13.

[57] Minchen Yu, Zhifeng Jiang, Hok Chun Ng, Wei Wang, Ruichuan Chen, and Bo Li. 2021. Gillis: Serving Large Neural Networks in Serverless Functions with Automatic Model Partitioning. In *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 138–148.

[58] Minchen Yu, Ao Wang, Dong Chen, Haoxuan Yu, Xiaonan Luo, Zhuohao Li, Wei Wang, Ruichuan Chen, Dapeng Nie, Haoran Yang, and Yu Ding. 2025. Torpor: GPU-Enabled Serverless Computing for Low-Latency, Resource-Efficient Inference. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 597–612. https://www.usenix.org/conference/atc25/presentation/yu

[59] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. 2025. BlitzScale: Fast and Live Large Model Autoscaling with O(1) Host Caching. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 275–293. https://www.usenix.org/conference/osdi25/presentation/zhang-dingyan

[60] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. SHEPHERD: Serving DNNs in the Wild. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 787–808.

[61] Wei Zhang, Weihao Cui, Kaihua Fu, Quan Chen, Daniel Edward Mawhirter, Bo Wu, Chao Li, and Minyi Guo. 2019. Laius: Towards latency awareness and improved utilization of spatial multitasking accelerators in datacenters. In *Proceedings of the ACM International Conference on Supercomputing*. 58–68.

[62] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.

[63] Zhuangzhuang Zhou, Yanqi Zhang, and Christina Delimitrou. 2023. AQUATOPE: QoS-and-Uncertainty-Aware Resource Management for Multi-stage Serverless Workflows. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 1–14.